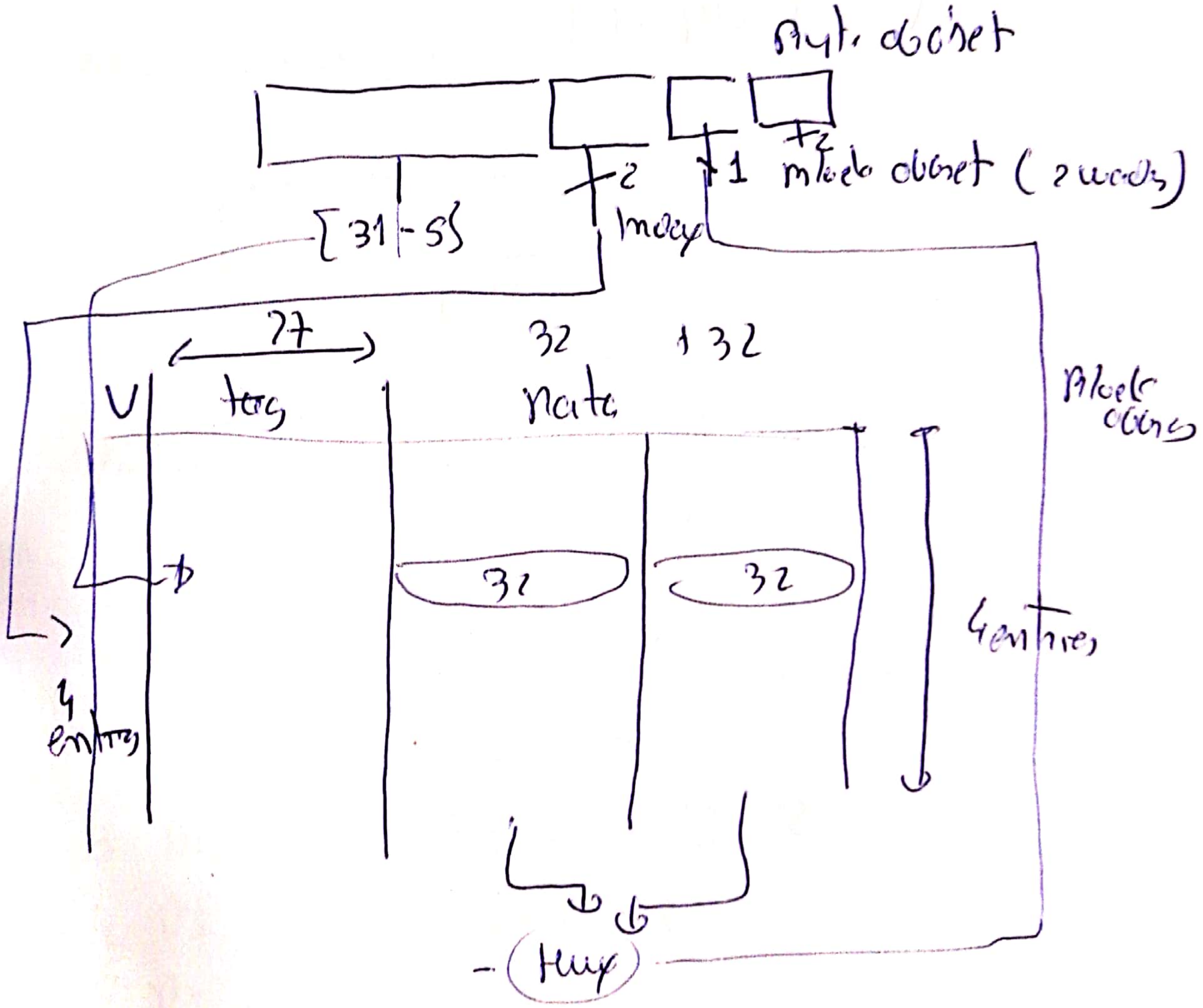
③ → Cache Directa
→ Write - through
→ 4 blocos **8 bytes** D cache

Bloco	Content								tag Address	V
0	2a	5d	04	aa	78	00	9c	23	7900002	1
1	16	62	34	66	7a	10	9f	a3	0000893	1
2	99	52	36	ee	88	60	3e	26	76ed001	1
3	42	15	90	ed	71	a6	3f	6d	65a6fff	0

a) show which address is generated for the cache access and mention the size of fields


Byte offset
[31-5]
2
Index
1
2
block offset (2 words)
27
32
32
V
tag
Data
Block access
32
32
4 entries
4 entries
Hit

To = 0x52 00 0044

|0000|0100|0100

Index Culida

0101 | 0010 | 0000 | 0000 | 0000 | 0000 | 0100 | 0 | 100

tag

mbelo

2	9	0	0	0	6	2

tag

Bloco 01

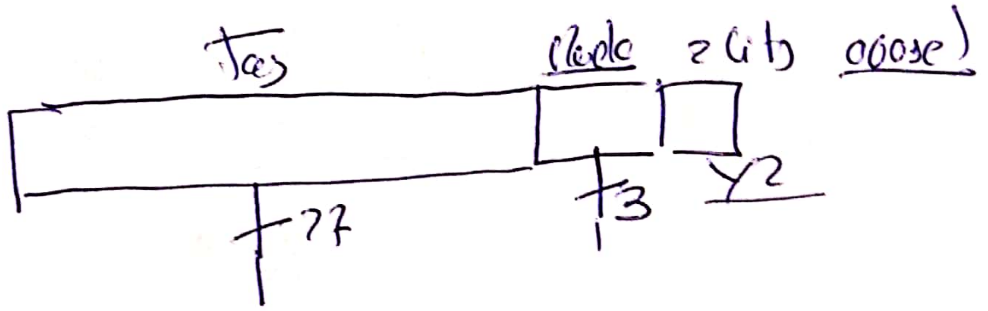
2a 5d 04 aa

⑥ 8 blocos 4 bytes

3 8 bits

	conteúdo	tag	V
	aa ff ee 33	123456a	0
	12 34 56 78	76Bd001	1
	88 60 3e 26	7fff d55	1
	71 a6 3f 6d	7fff d55	1
	34 ff 13 aa	07 f9a10	0
	78 00 5e 23	0000 B03	1
	79 10 9f 43	29000002	1
7	99 43 65 64	7f d1 d12	0

a) How address is decomposed? → $2^3 = 8$ blocos



b) Mostre, Justificando, quais acessos ocorrem na memória cache que segue a seguinte sequência

1 R1 ← 0x fa fa fa fa

2 R4 ← R3 + R2

3 R4 ← 0x 0f f3 22 10

4 R5 ← Mem[R4]

5 R5 ← R4 + R5

6 R6 ← 0x ff ff aa a8

7 Mem[R6+4] ← R1

① R_1 é 0xfafafafa

não há acesso memória cache,
não há acesso a memória

② R_4 é $R_3 + R_2$

não há acesso a memória,

③ R_4

não há acesso a memória

④ $R_5 \leftarrow \text{Mem}[R_4]$

Digitalizada com CamScanner

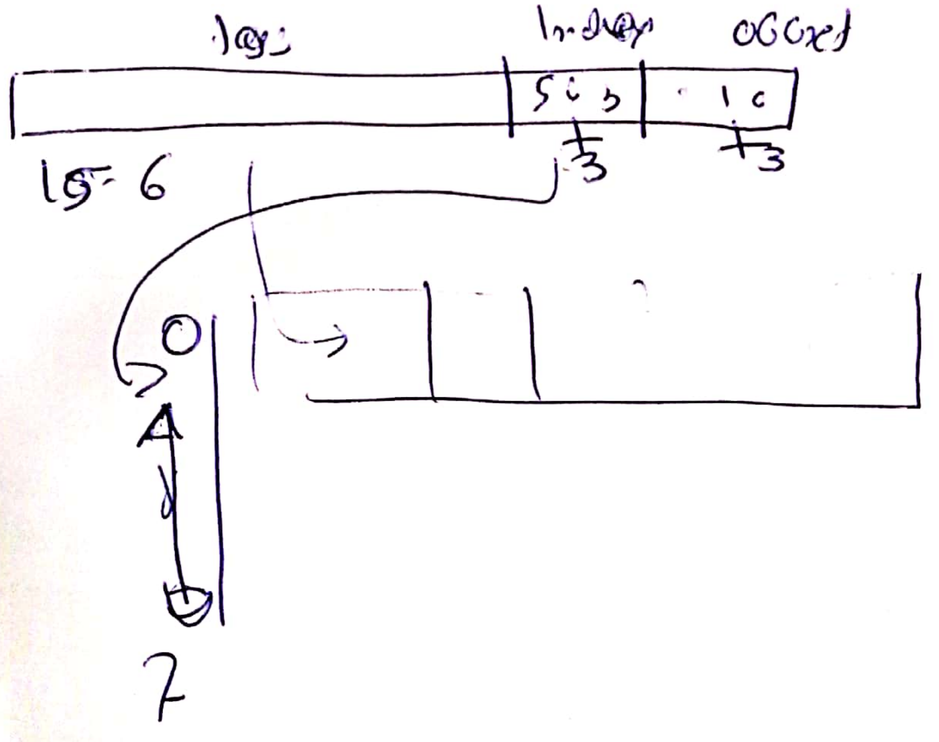
⑦ 16 bits address
8 blocos → 8 bytes

Block	Data	Tag	V	D
0	aa ce de bf 34 33 11 01	735	1	0
1	66 ad 45 4f af de 71 09	391	1	1
2	ce 34 a6 1f 56 cd ff ff	073	1	1
3	dd 67 72 26 32 56 72 71	198	0	1
4	ee 32 11 9f aa 6a a6 06	311	1	0
5	ff 10 00 04 01 02 03 04	778	0	0
6	11 03 41 32 ee dd ee ff	712	1	1
7	22 01 c5 01 05 06 07 08	387	0	1

9) address size → 16 bits
block size 8 bytes

$2^x = 8 = x = \underline{\underline{3}}$

$2^{\textcircled{3}}$ — blocos
8



⑨ 18 bits → endereço
Direct mapping
32 bits
16 janelas
write - through

Index = endereço memória mod (nº blocos)

Se nº blocos $= 2^6$ a índice é dado pelos 6 bits menos significativos

Índice bits $= 6$

Deslocamento nº bits $= w$

Etiqueta nº bits $= (18 - 6 - w - x)$

Exemplo
000000000000000000
11|1111|1111|1111|1111 18 bits
3 x x x x

Cache size memória $= 2^{(\text{Index} + \text{block offset} + 2)}$
$2^{(4 + 2)} = \underline{\underline{256}}$

c) n bits index $= 2^n = 16 \Leftrightarrow 4$ bits (Index)

tag = abc → $4 + 4 + 4 = 12$ bits (tag)

n total bits

tag size = total - $(n + m + 2)$

total n bits $= 2^n \times$ (block size + tag size + valid size)

block number = (block address) mod (N blocks)

index = 4 bits
tag = 12 bits

$\rightarrow \underline{2^{14}} \times (\overset{V}{1} + \overset{tag}{12} + \overset{wordsize}{32}) = \underline{\underline{720 \text{ bits}}}$

c) L -> leitura , E -> escrita

① 2df10 00|10|11|01|11|11|0001|0000 offset
0 B 7 C index 4 offset 8

① B7C tag B7E falta
4 index -> locata lê
V valida ① 0x6198fa34

② 23454 00|10|0011|0100|0101|01|01|00 offset
0 8 1 index ⑤

8B1 tag => falta
5 index -> lê da memoria principal e coloca no índice
0 = 0!
5 tag 8B1 e V=1

③ 3F7B0 00|11|1111|0111|1011|00|00
F D E index 12

FDE tag = FDF
12 index não falta
V

lê valor da memoria principal e coloca o FDE, index 12, V=1

④ 048e4   0000|0100|1000|11 1|00|01|001
1 2 3 index 1

123	tag
1	index
1	V

= dealth

Escreve 0x1212a606 na posição 1, V=1 e atualiza memória no endereço 048e4

⑤ 3f7d0   0011|1111|0111|11 01|0000
f d f index 4

1df	tag
4	index

→ para o index 4 a tag é diferente (67e). Não altera cache, colocar o valor 00001111 na memória principal na posição 3f7d0

⑥ 1dde8   0001|1101|1101|11 10|01,00
0 7 7 7 9

177	tag
9	index
V	= 1

no match

Não altera cache, tag é diferente (877)

Coloca 0xaaaa6606 em endereço 0x1dde8

⑩ Write back

4) Comple ~~d~~

Como igual cache - item

① 2df10

00|0|11|01|11|1 1|0001|0000
0 B 7 e index 4

B7e tag
4 index
v 1

Hit

le o valor 6198fa34 da memoria cache

②

B7e tag
4 index
v 1
d =0

write hit

como d=0 não é preciso fazer write back
o valor 33334444
é escrito na memoria cache no bloco 4
v=1, d=1, tag B7e

③ 10e64

00|01|0000|11|00|01|0|01|00
0 4 3 1 9 index
tag

431 tag
9 index
v=1
d=1

write miss

é preciso v=1, d=1 e o valor ffff efe é preciso ser escrito na memoria principal no endereço B77 + index e
o valor [illegible] e escrito em 431+9

④ 23454

|0010|0011|0100|0101|01|00
8 d 1 5 index

8d1 tag
5 index
V= 0
d= 1

read miss v=0

write back - memória.
leio o valor da memória
e coloca index 5, tag 8d1
V=1, d=0

⑤ 3F760

fde tag
8 index

v=1
d=1

0110 0000

0011|1111|0111|0110|0000
f d d 8

0011|1111|0111|0110|0000
8

Read hits

V=0 não a há write back
lê memória principal e
coloca no index
fde, 12

⑫ ⑪ dom...

(11) Dados $L1 \rightarrow$ Mem $\underline{70ns}$

36% acesso a memória

	Cache	% Miss	tempo acesso
P1	1KB	11,4%	0,62ns
P2	2KB	8.%	0,66ns

a) CPU Time = Instruction count × CPI × clock período

CPI = CPI perfect + CPI stall

$$\text{clock Rate}_{P1} = \frac{1}{0,62 \times 10^{-9}} \Rightarrow \underline{\underline{1,61\ GHz}}$$

$\underline{\text{tempo acesso}}$

$$\text{clock Rate}_{P2} = \frac{1}{0,66 \times 10^{-9}} = 1,52\ GHz$$

b) CPI avg acesso memória $\underline{2 \text{ casos}}$

tempo acesso

$$P1: \text{tempo memória acesso time} = 1.00 \times 0,62 + \underbrace{0,114}_{\%\ \underline{\underline{\text{miss}}}} \times 70$$
$$\approx 8,6\ ns$$

$$P2: \text{tempo méd acesso time} = 1.00 \times 0,66 + \underbrace{0,08}_{\%\ \underline{\underline{\text{miss}}}} \times 70$$
$$= 6,26\ ns$$

i) CPI = 1

P_1 miss penalty = $\frac{70}{0,62}$ = 113 cicles

of P_2 miss penalty = $\frac{70}{0,66}$ = $\underline{107}$ cicles

CPI stall instruction access$_{P_1}$ = $1 \times 0,114 \times 113 = 12.882$

CPI stall data access$_{P_1}$ = $0,3 \times 0,114 \times 113 = 4.38$

CPI$_{P_1}$ = CPI perfect + CPI stall data access
+ CPI stall ins access

CPI$_{P_1}$ = $1 + 12.882 + 4,38 = 18,262$

CPI stall instrution access$_{P_2}$ = $1 \times 0,08 \times 107 = 8,56$

CPI stall data access$_{P_2}$ = $0,34 \times 0,08 \times 107 =$
2,91

CPI$_{P_2}$ = $1 + 8,56 + 2,91 = 12,47$

$$\frac{18,262}{12,47} = \underline{P_2 \times 1,39\ P_1}$$

Digitalizada com CamScanner

⑫ Gepo = 1GHz

Taxa falha cache instruções = 5%
Taxa falha cache dados = 10%

Tempo memória principal = 80ns

Média load/store = 40% instruções

a) 1000 instruções

1000 instruções implicam 1000 acessos a I-cache e 400 acessos a D-cache → $\boxed{1000 \cancel{\times 0,4}}$

logo: I-cache → 1000 × 0,05 = 50 falhas em/1000
D-cache → 400 × 0,1 = 40 falhas em/1000

No total temos 90 falhas/1000 + 1000 ×0,4
ou seja 1400 acessos à memória

b) Para a mesma cache apresentar as mesmas características, significa a ocorrência de 90 falhas em 1400 acessos realizados. logo 90/1400 = 0,0643
$\underline{6,43\%}$

c) CPI perfect = 1,2

CPI = CPI perfect + CPI stall

CPI stall = Nacessos/Inst × Miss rate × Miss penalty (ciclos)

Miss penalty (ciclos) = $\frac{80ns}{1ns}$ = 80 ciclos

~~CPI stall~~ = $N_{acessos}$ acessos/Inst = 1 + 0,4 (LW and SW) = 1,4

1) Sem cache:

$CPI = 1.2 + \boxed{1.4} \times \overset{10}{\cancel{\times}} 80 = 113,$ ciclos / Instrução

Com cache:

$CPI = 1.2 + 0.0643 \times \boxed{1,4} \times 80 = 8.4$ ciclo / Instr)

$$\frac{\text{N acesso}}{\text{N Instr}} = \frac{1000 + 400}{1000}$$

$= \underline{1.4}$

⑬ Endereços 32 bits
4 words / bloco
4096 blocos = $\underline{4k}$ blocos

byte offset
4 byte 32 bits

a) • Mapeamento direto

4k blocos => 2^{12} blocos → index $\underline{12 \text{ bits}}$
word offset = $2^{②} = 4$ (para indexar 4 words/bloco)
byte offset = $2^{②} = 4$ (" " 4 bytes/word)

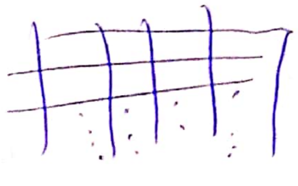
nº bits tag = 32 - 12 - 2 - 2 = 16 bits
nº bits total etiquetas = 16 × 4k = $\underline{64k}$

• Associativa c/ 2 vias

→ 2 blocos / entrada de cache

-12-1
$\boxed{11}$ bits para o index , 1 bit mais para a $\underline{\text{tag}}$
16+1 bit tag = $\underline{17}$
nº total tags = 17 × 4k = 68 kbits

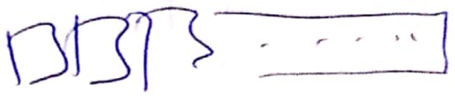
• Associatividade c/ 4 vias



→ 4 blocos / entrada cache

index 12 - 2 = 10 bits
tag 16 + 2 = 18 bits
nº total etiquetas = 18 × 4k = 72 kbits

- Associatividade total
  - → 4k blocos na única entrada
    - 0 bits index



$\text{tag} \quad 32 - \overset{\text{word sel}}{2} - \overset{\text{byte offset}}{2} = \underline{28 \text{ bits}}$

$n \text{ total bits tag} = 28 \times 4k = 112 \text{ kbits}$

(14) cache associativa 2 way

memória endereçável 64 KB → 16 bits

Dados tabela

dados: 8 x 8 bytes = 64 bytes

lru: 2 bits

valido 2 bits

(8) → 3 bits

word offset $2^{(3)} = 8$ index

tags = 16 - 3 - 2 = 11 bits

~~32+32 + 22 + 2+2 =~~

~~11 x 2 bytes = 22 bytes~~

total = 64 ~~+22 +2+2~~

$32 + \underbrace{32 + 11}_{tag1} + \underbrace{11}_{tag2} + 2 + 2 =$ ~~90 bytes~~ = 90 bits

b) 2 tags → 22 comparações de bits

e) 8567 → | 1000 | 0101 | 0110 | 0 | 0 | 11

tag index

4 2 B 1

tag = 42B ok

index = 1

V=0 não valido

F798 → | 1111 | 0111 | 1001 | 1000

index

7 B C 6

tag = 7Bc

index = 6

V = 1

ff

d)

° 663E   0110|0110|0011|1110
3 3 9 7

Tag = 331
Index = 7

Tag bait
não existe em cache

d) DE.

f) ° 01FF   0000|0001|1111|1111
index

° 831E   0100|0011|0001|11.00
7

O conteúdo de cache deveria ficar numa "via" diferente

9 1130 |0011|0000
4

9 1150 |01|0000
4

5A 71 |0111|0010
4

3 candidato 2

1) dirty
→ cache - through
→ write - back

→

Trata-se de uma memória cache do tipo write-back, pois memórias do tipo write-through não necessitam indicação modificação.

X

CPU clock = 1×10^9

CPI stall inst access = 1 × Miss Rate inst access × Miss penalty

CPI stall data access = 1000 stores / IC × Miss Rate data access × Miss penalty

CPI = CPI perfect + CPI stall inst access + CPI stall data access

CPU clock = 1×10^9

Miss rate inst. access = 0,05

Miss rate data access = 0,10

t acesso L1-Mem = 80ns

LoadStore / IC = 0,4

a) Calc on 1000 instructions

CPI stall inst access = 1 × 0,05 × Miss penalty

CPI stall data access = 0,4 × 0,1 × Miss penalty

Miss penalty = $\frac{80}{1ns}$

⑮

32bits

Tempo Mem = 80ns

C1 64 KiB
64 palavras/bloco
Direct Mapping
tacerto = 1ns
~~Falha~~ taxa falha = 10%
write-back

C2: 512 KiB
128 palavras/bloco
4 vias
tacerto = 4ns
taxa falha = 25%
write-through
LRU -> least recently used

a) Determinar valor etiqueta quando acede
0x25 031 968

0010|0101|0000|0011|0001|1001|01|10|1|000

2 9 tag 0 3 index word offset byte offset

byte offset = 4 bytes / 32 bits

tag = 203

Nº blocos

$$\frac{64\,KB}{64 \times 4} = \frac{2^{10}}{2^2} = 256 \text{ blocos} \Rightarrow \text{Index} = 8 \text{ bits}$$

64 words / bloco ⇒ word offset 2^6 → 6 bits

L2:

Nº entradas cache = $\frac{512\ KB}{4 \times 128 \times 4B} = \frac{2^{10}}{2^2} = 256$

⇒ Indice 8 bits

128 words / bloco ⇒ 7 bits word offset

~~|0000|0011|0001|1001|0110|1000~~

4 GiB

0010 |0100 |0000 |0011 |0001 |1001 |0110 |1000

1 2 8 1 Index word offset

etq 1 = 0x025 03

etq 2 = 0x 1281

6) 1000 instruções

10% stores

20% loads

30% branch

40% aritméticas

acessos memória (10 + 20) = 30%

(15) 32 bits

tempo acesso memória principal = 80ns

L1: 64 KiB
64 words/bloco
latência = 1ns
taxa falha = 10%
write-back

L2: 512KiB
128 words/bloco
4 vias
latência = 4ns
taxa falha = 25%
write-through
LRU → least recent uses

a) 0x25031968

L1:

$$\underbrace{\overset{2}{0010}|\overset{5}{0101}|\overset{0}{0000}|\overset{3}{0011}|}_{\text{tag}}\underbrace{0001|1001|}_{\text{index}}\underbrace{0110|10}_{\text{word offset}}\underbrace{00}_{\text{byte offset}}$$

index → 64K → 2^8

word offset 6 bits (64 words)

nº entradas cache (index)

$$\frac{64\,KiB}{64 \times 4B/\text{bloco}} = \frac{2^{10}}{2^2} = 256 \text{ blocos entradas} = 8 \text{ bits}$$

64 words/bloco word offset 6 bits

$$\underbrace{\overset{1}{0010}|\overset{2}{0101}|\overset{P}{0000}|\overset{1}{0011}|}_{\text{tag}}\underbrace{0001|1001|}_{\text{8 bits index}}\underbrace{0110|10}_{\text{7 word offset}}\underbrace{00}_{\text{byte offset}}$$

$$n^{\circ} \text{ entradas cache} = \frac{512\,KiB}{4 \times 128 \times 4B/\text{entrada}} = \frac{2^{10}}{2^2} = 256$$

index = 8 bits

128 word/bloco = 7 bits word offset

log L1 = 0x2503
log L2 = 0x1281

c) 10% stores
20% load
30% branches
40% ~~branches~~ aritméticas

ex 1000 inst ⇒ 1000 acessos L1
⇒ 1000 × 0.1 acessos L2
⇒ 1000 × 0.1 × 0.25 acessos Mem
falha L1 falha L2

25
Mem

ex stores 1000 × 0.1 = 100 acessos L1
⇒ 1000 . 0.1 × 0.1 = 10 acessos L2
write-through hit/miss acede
a memória principal sempre

ex loads 1000 × 0.2 = 200 acessos L1
1000 × 0,2 × 0,1 = 20 acessos L2
1000 × 0,2 × 0,1 , 0,25 5 acessos Mem

Total acessos memória principal

25 + 10 + 5 = 40 acessos Mem

c) $t_{medio} = t_{acerto_{L1}} + tax_{falha_{L1}} \times (t_{acerto_{L2}} + tax_{falha_{L2}} \times Penalty)$